

Trabalho 1: Máquinas de Turing

Maria Eduarda Teixeira Costa e Sofia Gazolla da Costa Silva

Departamento de Informática e Estatística - Universidade Federal de Santa Catarina

Teoria da Computação

Professora Jerusa Marchi

Junho 2026

1 $L = \{a^i b^j c^k \mid i, j, k \in Ne\ i \times j = k\}$

1.1 Algoritmo em Alto Nível

Entrada: uma palavra da forma $a^i b^j c^k$.

1. Verificar se a entrada está no formato $a^i b^j c^k$. Caso contrário, rejeitar.
2. Selecionar o primeiro símbolo a ainda não processado e marcá-lo como A.
3. Para o símbolo A selecionado:
   (a) Procurar um símbolo b não marcado e marcá-lo como B.
   (b) Deslocar-se até a região dos símbolos c.
   (c) Procurar um símbolo c não marcado e marcá-lo como C.
   (d) Retornar à região dos símbolos b.
   (e) Repetir os passos anteriores até que todos os símbolos b tenham sido marcados.
4. Após todos os símbolos b terem sido utilizados:
   (a) Restaurar todos os símbolos B para b.
   (b) Retornar ao início da fita.
   (c) Procurar outro símbolo a não marcado.

(d) Caso exista, marcá-lo como A.

(e) Repetir o Passo 3.

5. Quando não houver mais símbolos a não marcados:

(a) Verificar a região dos símbolos c.

(b) Se existir algum símbolo c não marcado, rejeitar.

(c) Caso todos os símbolos c estejam marcados como C, aceitar.

A ideia da máquina é consumir um símbolo c para cada combinação possível entre um símbolo a e um símbolo b. Dessa forma, ao término da execução, a palavra será aceita se, e somente se, a quantidade de símbolos c for igual ao produto entre as quantidades de símbolos a e b, isto é, quando $k = i \times j$.

1.2 Multiple Run da Máquina

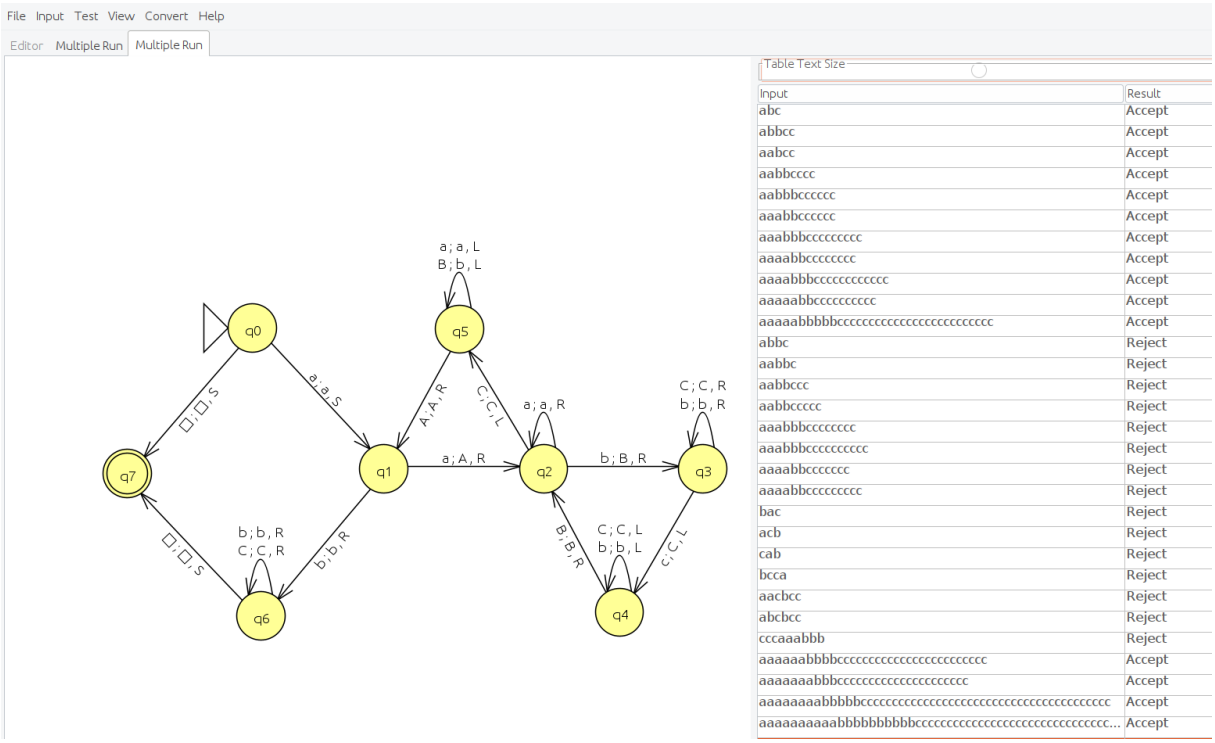


Figure 1: Multiple Run da Questão 1

Table 1: Casos de teste utilizados para validação da máquina

(i,j,k)	**Resultado**
(1,1,1)	Accept
(1,2,2)	Accept
(2,1,2)	Accept
(2,2,4)	Accept
(2,3,6)	Accept
(3,2,6)	Accept
(3,3,9)	Accept
(4,2,8)	Accept
(4,3,12)	Accept
(5,2,10)	Accept
(1,2,1)	Reject
(2,2,1)	Reject
(2,2,3)	Reject
(2,2,5)	Reject
(3,3,8)	Reject
(3,3,10)	Reject
(4,2,7)	Reject
(4,2,9)	Reject
Formato inválido (bac)	Reject
Formato inválido (acb)	Reject
Formato inválido (cab)	Reject
Formato inválido (bcca)	Reject
Formato inválido (aacbcc)	Reject
Formato inválido (abcbcc)	Reject
Formato inválido (cccaaabbb)	Reject
(5,5,25)	Accept
(6,4,24)	Accept
(7,3,21)	Accept
(8,5,40)	Accept
(10,10,100)	Accept

2 $L = \{\#x_1\#x_2\#...\#x_n \mid x_i \in 0,1^* e\ x_i \neq x_j\ para\ cada\ i \neq j\}$

2.1 Algoritmo em Alto Nível

Entrada: Uma fita contendo múltiplos segmentos $x_i, x_j, ...$ separados pelo delimitador #.

1. **Marcação de Segmento Base (x_i):** A máquina substitui o delimitador # inicial por um símbolo X, marcando o segmento atual x_i como o termo base para comparação.
2. **Cópia para Comparação:**
   (a) A máquina copia o conteúdo de x_i da Fita 1 para a Fita 2 até encontrar o próximo #.
   (b) A máquina copia o conteúdo de x_j (o segmento seguinte) para a Fita 3.

3. **Rebobinamento e Comparação:**

   (a) Ambas as fitas 2 e 3 são retornadas ao início.

   (b) Inicia-se a varredura comparativa:

   - Se houver qualquer diferença entre os bits ou diferença de tamanho, a configuração é considerada válida e a máquina prossegue.
   - Caso as sequências sejam idênticas até o final, a máquina rejeita a entrada.

4. **Iteração sobre x_j:**

   (a) A Fita 3 é limpa. A máquina avança para o próximo segmento na Fita 1 e o copia para a Fita 3.

   (b) O laço de comparação é repetido para cada x_j disponível. Se não houver mais x_j para comparar com o x_i atual, a máquina avança.

5. **Iteração sobre x_i:**

   (a) A Fita 2 é limpa para preparar a leitura de um novo segmento x_i.

   (b) A máquina rebobina a Fita 1, marca o próximo # como X e reinicia o processo de comparação de um novo x_i com todos os seus sucessores x_j.

6. **Finalização:** Caso não existam mais segmentos para comparar, a máquina encerra o processo no estado de aceitação.

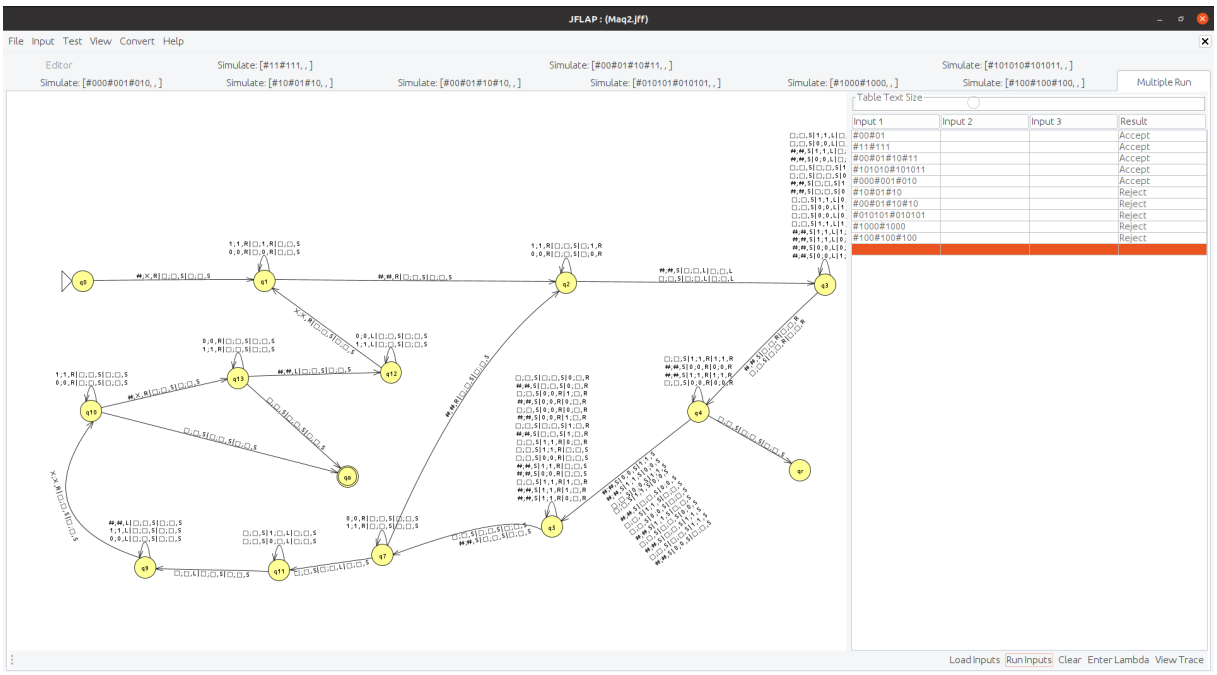


Figure 2: Multiple Run da Questão 2

Table 2: Casos de teste utilizados para validação da máquina

Entrada	**Resultado**
#00#01	Accept
#11#111	Accept
#00#01#10#11	Accept
#101010#101011	Accept
#000#001#010	Accept
#10#01#10	Reject
#00#01#10#10	Reject
#010101#010101	Reject
#1000#1000	Reject
#100#100#100	Reject

3 Funções

(a) **Série de Fibonacci:** A máquina recebe como entrada uma sequência de símbolos que representa n (representação unária - como X ou a). Ao término, deve constar na fita uma sequência de símbolos que indica o valor do n-ésimo termo, ou seja $Fibonacci(n)$. Faça com que a máquina, em seus primeiros passos de computação grave as sementes 0 e 1 e proceda o cálculo iterativo da série até n.

Algoritmo em alto nível:

1. A máquina percorre a entrada n na Fita 1 até o final e inicializa as sementes, gravando o valor do primeiro termo na Fita 2 e mantendo a Fita 3 vazia.
2. O ciclo de cálculo começa com a Fita 1 recuando um símbolo. Os valores da Fita 3 (termo anterior) e da Fita 2 (termo atual) são copiados sequencialmente para a Fita 4, que atua como acumulador da soma.
3. As variáveis são atualizadas: a Fita 3 recebe o valor antigo da Fita 2, e a Fita 2 recebe o resultado da nova soma armazenado na Fita 4.
4. Os cabeçotes das fitas auxiliares são rebobinados ao início e a Fita 4 apaga o seu próprio conteúdo para preparar a próxima iteração.
5. O processo repete-se até que o cabeçote da Fita 1 encontre o espaço vazio inicial. O valor do n-ésimo termo é então copiado da Fita 2 para a Fita 1, e a máquina transita para o estado de aceitação.

Printscreen da máquina:

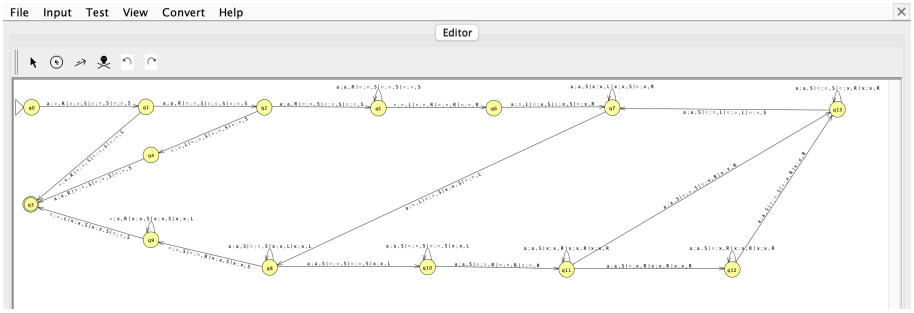


Figure 3: MT do Algoritmo de Fibonacci

(b) **Algoritmo de Euclides para o Máximo Divisor Comum:** A máquina recebe como entrada uma sequência de símbolos representando n e m em representação unária. Ao término, a fita deve conter o $MDC(n, m)$.

Algoritmo em alto nível:

1. A máquina lê a entrada original $n\#m$ na Fita 1, copia o valor de n para a Fita 2 e o valor de m para a Fita 3, retornando os cabeçotes ao início.
2. As Fitas 2 e 3 são lidas simultaneamente, substituindo os símbolos por marcadores temporários para comparar os tamanhos sem perder os dados.
3. O valor menor é subtraído do maior. Se $n > m$, a máquina apaga a diferença na Fita 2 e restaura a Fita 3. Se $m > n$, apaga a diferença na Fita 3 e restaura a Fita 2.
4. Os cabeçotes pulam os espaços vazios e são realinhados no início dos símbolos restantes para reiniciar o loop de comparação.
5. O processo se repete até as duas fitas se igualarem ($n = m$). A máquina limpa a Fita 1, copia o valor do MDC para ela e transita para o estado de aceitação.

Printscreen da máquina:

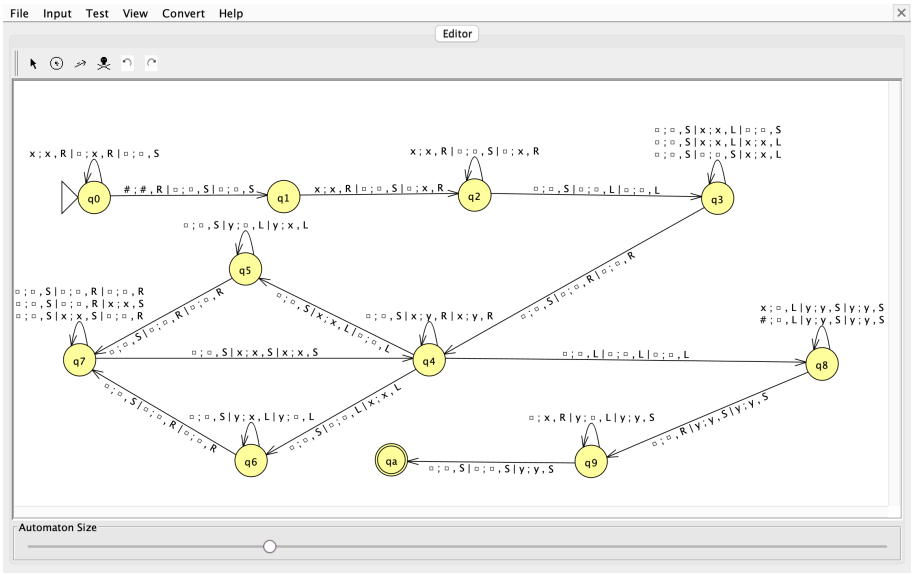


Figure 4: MT do Algoritmo de Euclides para cálculo de MDC

(c) **Triângulo de Pascal:** A máquina recebe como entrada um par (linha, coluna) e retorna o elemento correspondente no triângulo.

Algoritmo em alto nível:

1. A máquina lê a entrada e inicializa os contadores: a linha alvo (n) e a coluna alvo (k) controlam o número de iterações. As sementes iniciais (Linha 0) são gravadas nas fitas.
2. O cálculo iterativo utiliza duas fitas dedicadas para gerenciar as linhas: uma fita atua como memória mantendo os blocos da linha anterior (separados por marcadores como #), enquanto a outra constrói a nova linha.
3. A cada passo da construção, a máquina utiliza uma fita auxiliar de rascunho para ler dois blocos adjacentes da linha anterior, somá-los ($C(n-1,k-1)+C(n-1,k)$) e gravar o novo bloco resultante na fita da linha atual.
4. Ao finalizar a construção de uma linha completa, os cabeçotes das fitas de trabalho são rebobinados, o conteúdo da nova linha passa a ser a "linha anterior" para o próximo ciclo, e o contador de iterações de linha é atualizado.
5. Esse processo repete-se até que a linha n seja totalmente gerada. A máquina então utiliza o contador k para navegar pelos blocos separados por marcadores na fita da linha gerada, isola o elemento exato da coluna desejada, copia-o para a fita

principal e transita para o estado de aceitação.

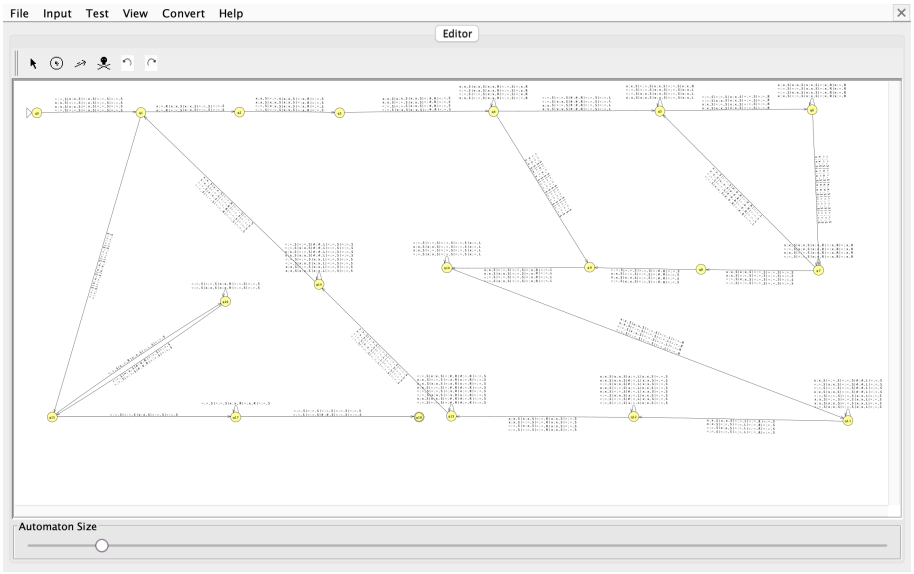


Figure 5: MT do Algoritmo do Triângulo de Pascal